Nmap & Scapy Lab Documentation

Objectives
Reproduce and document Nmap scans and Scapy packet manipulation exercises, as covered in class. Summarize the process, findings, and reflect on the application of these tools in real-world cybersecurity.

Lab Environment

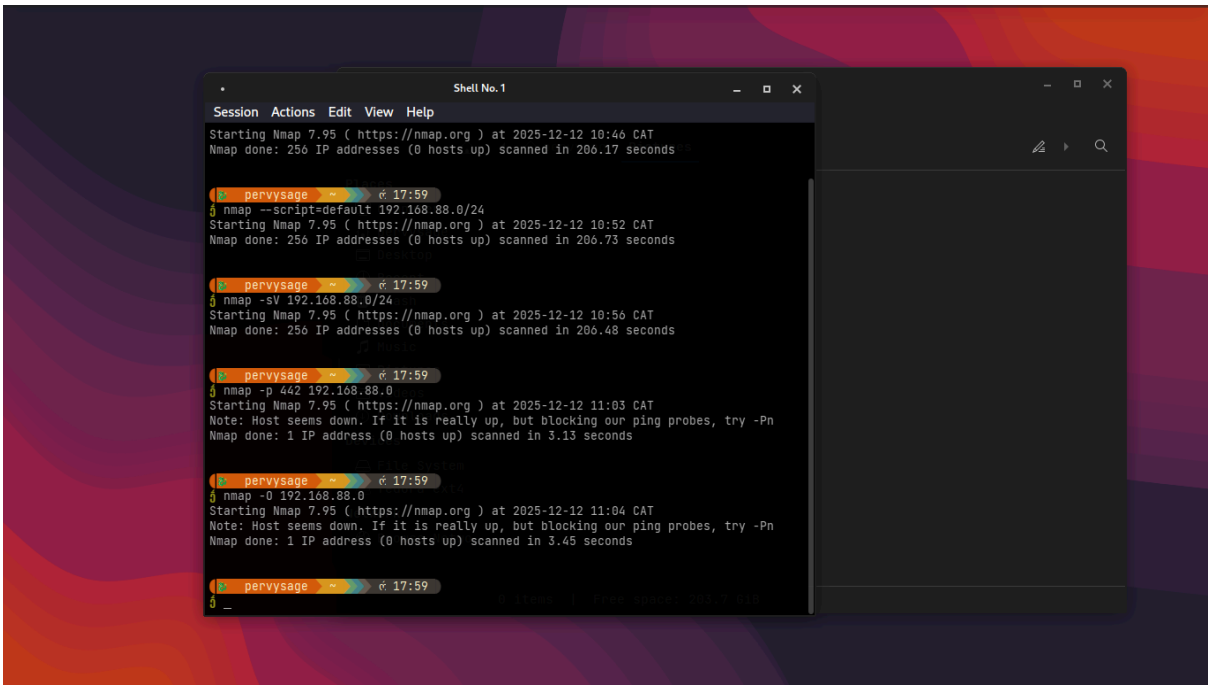
Operating System:*Kali-Linux*
Primary Tools:
- Nmap 7.95
- Python 3.x
- Scapy 2.6.1

Network Setup:*home*

Nmap Lab Commands Executed
sh
nmap --script=default 192.168.88.0/24
nmap -sV 192.168.88.0/24
nmap -p 442 192.168.88.0
nmap -O 192.168.88.0

Results & Screenshot

(Nmapscans.png)



Summary & Explanations:
- `--script=default` executes the default scripts for safe and insightful scanning.
- `-sV` detects service versions on open ports.

- `-p 442` checks if port 442 is open on the target.
- `-O` attempts OS detection.

In the screenshot above, most hosts appear down. This can happen due to firewalls or network policy. In real penetration tests, it’s common to use additional flags (like `-Pn`) when hosts block ping requests.

Scapy Lab
Packet Crafting & Sending

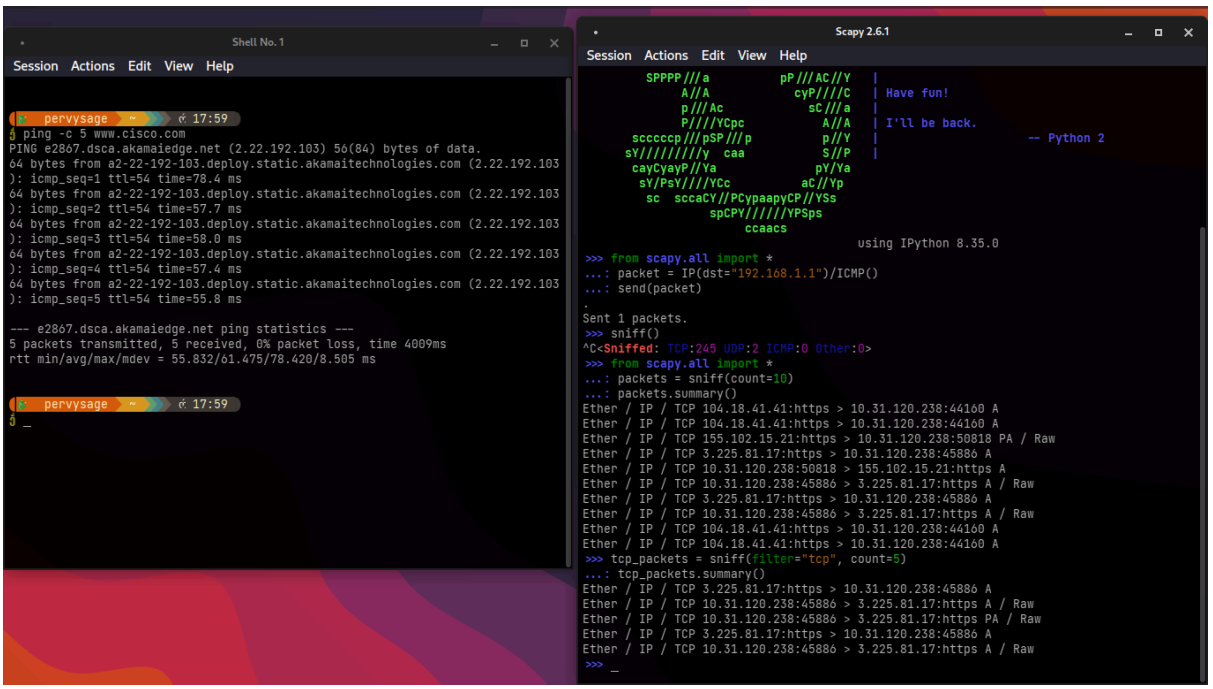
Python Code:
```python
from scapy.all import
packet = IP(dst="192.168.1.1")/ICMP()
send(packet)
```
This script sends a custom ICMP echo request ("ping") to the target IP address.

Sniffing Packets & TCP Filtering

Python Code:
```python
from scapy.all import *
packets = sniff(count=10)
packets.summary()

tcp_packets = sniff(filter="tcp", count=5)
tcp_packets.summary()
```
This code captures 10 packets from the network, then filters and summarizes 5 TCP packets.

Results & Screenshot
(Scapyscans.png)



Summary & Explanations:
- `sniff(count=10)` lets us observe live network traffic.
- `.summary()` displays brief info about each captured packet.
- Filtering by TCP demonstrates protocol-specific monitoring, essential for network troubleshooting and penetration testing.

Real-World Ping Comparison
Command:
```sh
ping -c 5 www.cisco.com
```
Tested basic ICMP functionality.

(Screenshot for this step is combined in the previous image on the left.)

Results & Learnings

- Nmap is a versatile tool for discovering devices, scanning open ports, identifying services, and basic OS fingerprinting.
- Scapy gives deeper control, allowing custom-crafted packets and flexible, programmatic packet sniffing.
- Real-world issues (like hosts refusing ping) were encountered, highlighting the need for method adaptation (eg, `-Pn` flag).
- Gained confidence using command-line utilities and Python scripting for network security tasks.

Reflection

See my post for a summary of my experience, challenges, and connections to real cybersecurity practice:
👉
[https://www.linkedin.com/posts/wongani-nkosi-36b53b297_parocyber-rolandmawuliawuku-activity-7405174191014084608-GxSW?utm_source=share&utm_medium=member_desktop&rcm=ACoAAEfN60MB2cRurVzRPeCmTr_c6CHUqTW3MvA]

References

- [Nmap Official Documentation](https://nmap.org/book/man-briefoptions.html)
- [Scapy Documentation](https://scapy.readthedocs.io/en/latest/)

Notes:
All screenshots were taken during hands-on sessions. For privacy, some target IPs were anonymized.